

Contents

Bug-Free Programming Guide	1
Best Practices for the Binary Decomposition Interface (BDI) Project	1
Executive Summary	1
Table of Contents	2
Memory Management Best Practices	2
Concurrency and Synchronization Best Practices	7
Error Handling Best Practices	12
I/O and File System Best Practices	16
Algorithm Design Best Practices	18
Code Quality and Maintainability	21
Testing and Validation Strategies	24
Security Best Practices	28
Performance Optimization Guidelines	31
Code Review Checklist	33
Conclusion	33

Bug-Free Programming Guide

Best Practices for the Binary Decomposition Interface (BDI) Project

Guide Version: 1.0

Last Updated: October 10, 2025

Project: The Binary Decomposition Interface

Audience: All BDI developers and contributors

Executive Summary

This comprehensive programming guide consolidates best practices, lessons learned, and guidelines derived from analyzing and fixing 35+ critical bugs across 165 pull requests in the BDI project. The guide is organized by category with specific examples of correct vs. incorrect patterns.

Guide Objectives

- Prevent common bugs** through clear guidelines
- Promote memory safety** in all code
- Ensure concurrency correctness** in multi-threaded code
- Maintain data integrity** across operations
- Establish coding standards** for consistency

Quick Reference

- Memory Management
 - Concurrency & Synchronization
 - Error Handling
 - I/O Operations
 - Algorithm Design
 - Testing Strategies
-

Table of Contents

1. Memory Management Best Practices
 2. Concurrency and Synchronization Best Practices
 3. Error Handling Best Practices
 4. I/O and File System Best Practices
 5. Algorithm Design Best Practices
 6. Code Quality and Maintainability
 7. Testing and Validation Strategies
 8. Security Best Practices
 9. Performance Optimization Guidelines
 10. Code Review Checklist
-

Memory Management Best Practices

Rule MM-1: Always Validate Array Bounds

Principle: Check bounds for all array accesses, especially when accessing multiple elements.

WRONG:

```
// Bug: Accesses i+2 without checking
for (size_t i = 0; i < array_size - 1; i++) {
    if (array[i] == X && array[i+2] == Y) { // Out of bounds when i == size-2
        // ...
    }
}
```

CORRECT:

```
// Always check bounds for multi-element access
for (size_t i = 0; i < array_size - 1; i++) {
    if (i + 2 < array_size && // Explicit bounds check
        array[i] == X && array[i+2] == Y) {
        // ...
    }
}
```

Key Points: - Loop condition `i < n-1` does NOT guarantee `i+k < n` for `k > 1` - Always add explicit bounds check: `if (i + k < size)` - Use assertions in debug builds: `assert(i + k < size)` - Test with small arrays (`size < k`)

Rule MM-2: Store Original Pointer for Aligned Allocations

Principle: Never lose the original pointer when returning aligned addresses.

WRONG:

```
void* alloc_aligned(size_t size, size_t alignment) {
    void* ptr = malloc(size + alignment);
    uintptr_t aligned = ((uintptr_t)ptr + alignment - 1) & ~(alignment - 1);
    return (void*)aligned; // Original pointer lost!
}

void free_aligned(void* ptr) {
```

```
    free(ptr); // Frees wrong address - corruption!
}
```

CORRECT:

```
void* alloc_aligned(size_t size, size_t alignment) {
    // Allocate extra space for metadata
    size_t alloc_size = size + alignment + sizeof(void*) + sizeof(size_t);
    void* original = malloc(alloc_size);

    // Calculate aligned address after metadata
    uintptr_t addr = (uintptr_t)original + sizeof(void*) + sizeof(size_t);
    uintptr_t aligned = (addr + alignment - 1) & ~(alignment - 1);

    // Store metadata before aligned address
    void** ptr_storage = (void**)(aligned - sizeof(void*) - sizeof(size_t));
    size_t* size_storage = (size_t*)(aligned - sizeof(size_t));

    *ptr_storage = original;
    *size_storage = alloc_size;

    return (void*)aligned; // Original stored
}

void free_aligned(void* ptr) {
    // Retrieve original pointer and size
    void** ptr_storage = (void**)((uintptr_t)ptr - sizeof(void*) - sizeof(size_t));
    size_t* size_storage = (size_t*)((uintptr_t)ptr - sizeof(size_t));

    void* original = *ptr_storage;

    free(original); // Free original pointer
}
```

Key Points: - Store original pointer before aligned address - Store allocated size for proper deallocation - Provide separate `free_aligned()` function - Document usage requirements clearly - Follow patterns from `_aligned_malloc()`, `aligned_alloc()`

Memory Layout:

```
[original_ptr] [alloc_size] [padding...] [aligned_address]
                                     ^-- returned to user
```

Rule MM-3: Share Reference Counts for Shared Resources

Principle: Use one reference count per shared resource, not one per accessor.

WRONG:

```
// Copy-on-write memory region - BUGGY
MemoryRegion* child_region = malloc(sizeof(MemoryRegion));
child_region->base = parent_region->base; // Same physical memory
child_region->size = parent_region->size;

atomic_init(&child_region->ref_count, 1); // Independent counter!
atomic_fetch_add(&parent_region->ref_count, 1);
```

*// Problem: When child exits, it decrements its own counter to 0
// and frees the memory, but parent still thinks it owns it!*

CORRECT:

```
// Share the same region descriptor
MemoryRegion* shared_region = parent_region;

// Increment refcount for child
atomic_fetch_add(&shared_region->ref_count, 1, memory_order_relaxed);

// Both parent and child point to same MemoryRegion
// Refcount tracks total number of references
// Memory freed only when refcount reaches 0

// In parent:
parent->memory_region = shared_region;

// In child:
child->memory_region = shared_region;
```

Key Points: - One refcount per resource, not per accessor - Increment on share, decrement on release - Free only when refcount reaches 0 - Use atomic operations for thread safety - Test with multiple sharers exiting in different orders

Rule MM-4: Track Allocations Individually for Rollback

Principle: Don't assume memory contiguity without explicit guarantees.

WRONG:

```
void* alloc_pages(size_t count) {
    void* first_page = alloc_page();

    for (size_t i = 1; i < count; i++) {
        void* page = alloc_page();
        if (page == NULL) {
            free_pages(first_page, i); // Assumes pages are contiguous!
            return NULL;
        }
    }
    return first_page;
}
```

CORRECT:

```
void* alloc_pages(size_t count) {
    // Use stack array for small allocations, heap for large
    #define MAX_STACK_PAGES 64
    void* stack_pages[MAX_STACK_PAGES];
    void** pages = (count <= MAX_STACK_PAGES) ? stack_pages :
        (void**)malloc(count * sizeof(void*));

    if (pages == NULL && count > MAX_STACK_PAGES) {
        return NULL;
    }
}
```

```

}

// Track each page individually
for (size_t i = 0; i < count; i++) {
    pages[i] = alloc_page();

    if (pages[i] == NULL) {
        // Free each page individually
        for (size_t j = 0; j < i; j++) {
            free_page(pages[j]); // Free individually
        }
        if (count > MAX_STACK_PAGES) free(pages);
        return NULL;
    }
}

void* first_page = pages[0];
if (count > MAX_STACK_PAGES) free(pages);
return first_page;
}

```

Key Points: - Never assume contiguity without explicit guarantee - Track each allocation individually - Use stack allocation for small counts (optimization) - Clean up properly on failure - Test with NUMA fallback scenarios

Rule MM-5: Validate Address Ranges Match Capacity

Principle: Ensure address ranges cover all allocated resources.

WRONG:

```

// NUMA node setup - BUGGY
#define PAGES_PER_NODE 1048576 // 1M pages = 4 GB

numa_info[i].base_addr = (uint64_t)i * (1ULL << 30); // 1 GB per node
numa_info[i].end_addr = numa_info[i].base_addr + (1ULL << 30);

// Problem: 4 GB of pages but only 1 GB address range!
// After 262,144 pages (1 GB), addresses fall outside range

```

CORRECT:

```

// Calculate required address space
#define PAGES_PER_NODE 1048576 // 1M pages
#define PAGE_SIZE 4096
#define SPACE_PER_NODE (PAGES_PER_NODE * PAGE_SIZE) // 4 GB

numa_info[i].base_addr = (uint64_t)i * SPACE_PER_NODE;
numa_info[i].end_addr = numa_info[i].base_addr + SPACE_PER_NODE;

// OR using bit shifts:
numa_info[i].base_addr = (uint64_t)i * (1ULL << 32); // 4 GB per node
numa_info[i].end_addr = numa_info[i].base_addr + (1ULL << 32);

```

Key Points: - Calculate total memory requirements: $\text{count} \times \text{size}$ - Ensure address ranges match capacity
 - Use bit shifts carefully: $1\text{ULL} \ll 32 = 4\text{ GB}$ - Validate: $\text{pages} \times \text{page_size} \quad \text{end_addr} - \text{base_addr}$

Rule MM-6: Round Aligned Allocation Sizes

Principle: Ensure allocation sizes are multiples of alignment.

WRONG:

```
// C17 violation: size must be multiple of alignment
size_t total_size = sizeof(ring_t) + capacity * sizeof(void*);
// For capacity=16: 80 + 128 = 208 bytes (not multiple of 64!)

ring_t* ring = aligned_alloc(64, total_size); // May fail or UB!
```

CORRECT:

```
// Calculate size
size_t total_size = sizeof(ring_t) + capacity * sizeof(void*);

// Round up to multiple of alignment (ceiling division)
total_size = ((total_size + alignment - 1) / alignment) * alignment;

ring_t* ring = aligned_alloc(64, total_size); // Size is multiple of 64
```

Formula for ceiling division:

```
// Round up n to multiple of m:
rounded = ((n + m - 1) / m) * m;

// OR using bit operations for powers of 2:
rounded = (n + (m - 1)) & ~(m - 1);
```

Key Points: - C17 §7.22.3.1: size must be multiple of alignment - Use ceiling division formula - Validate alignment is power of 2 (for bit operation method) - Minimal overhead (at most alignment-1 bytes)

Rule MM-7: Handle Data Spanning Boundaries

Principle: Always account for structures that span page/sector boundaries.

WRONG:

```
// Read inode - BUGGY
uint8_t buf[512]; // Single sector
read_sector(device, sector, buf, 1);

uint32_t offset = inode_offset % 512; // e.g., 450
memcpy(&inode, buf + offset, sizeof(inode));
// If sizeof(inode) = 128, copies from buf[450] to buf[577]
// But buf only has 512 bytes!
```

CORRECT:

```
// Calculate sectors needed
uint32_t sector_offset = inode_offset % 512;
uint32_t sectors_needed = (sector_offset + sizeof(inode) + 511) / 512;

// Allocate buffer for all needed sectors
```

```

uint8_t buf[sectors_needed * 512];

// Sanity check
if (sectors_needed > MAX_SECTORS) {
    return -EINVAL;
}

// Read all necessary sectors
read_sector(device, sector, buf, sectors_needed);

// Now safe to copy
memcpy(&inode, buf + sector_offset, sizeof(inode)); //

```

Key Points: - Calculate exact buffer size needed - Read multiple sectors/pages if necessary - Add sanity checks (e.g., max 2 sectors for inode) - Test with structures at various offsets - Use AddressSanitizer to detect overruns

Concurrency and Synchronization Best Practices

Rule CS-1: Use Atomic Fetch Operations for Read-Modify-Write

Principle: Never split atomic read-modify-write into separate operations.

WRONG:

```

// Non-atomic RMW sequence - race condition!
uint64_t head = atomic_load(&queue->head, memory_order_acquire);

// ... process ...

atomic_store(&queue->head, head + 1, memory_order_release); // Not atomic!

// Problem: Another thread can modify head between load and store

```

CORRECT:

```

// Atomic fetch-and-add
uint64_t head = atomic_fetch_add(&queue->head, 1, memory_order_acq_rel);

// head now contains the OLD value (before increment)
// increment is guaranteed atomic

```

Available Atomic Operations:

```

atomic_fetch_add()    // Atomically: old = *ptr; *ptr += val; return old;
atomic_fetch_sub()    // Atomically: old = *ptr; *ptr -= val; return old;
atomic_fetch_or()     // Atomically: old = *ptr; *ptr |= val; return old;
atomic_fetch_and()    // Atomically: old = *ptr; *ptr &= val; return old;
atomic_fetch_xor()    // Atomically: old = *ptr; *ptr ^= val; return old;

```

Key Points: - Use atomic fetch operations for RMW sequences - Returned value is the OLD value (before modification) - Use appropriate memory ordering (usually acq_rel) - Test with multiple concurrent threads - Verify no races with ThreadSanitizer

Rule CS-2: Use CAS for Conditional Modifications

Principle: Use Compare-And-Swap when modification depends on current value.

WRONG:

```
// Pre-increment before checking - creates false full queue!
uint64_t head = atomic_fetch_add(&queue->head, 1, memory_order_acq_rel);
uint64_t tail = atomic_load(&queue->tail, memory_order_acquire);

if (head >= tail) {
    // Undo - but damage already done!
    atomic_fetch_sub(&queue->head, 1, memory_order_relaxed);
    return NULL;
}

// Between fetch_add and fetch_sub, head > tail
// Concurrent enqueue sees full queue and drops tasks!
```

CORRECT:

```
// CAS loop: only increment if condition met
while (1) {
    uint64_t head = atomic_load(&queue->head, memory_order_acquire);
    uint64_t tail = atomic_load(&queue->tail, memory_order_acquire);

    // Check condition BEFORE modification
    if (head >= tail) {
        return NULL; // Queue empty
    }

    // Try to atomically increment - only if head unchanged
    if (atomic_compare_exchange_weak(&queue->head, &head, head + 1,
                                    memory_order_acq_rel,
                                    memory_order_acquire)) {
        break; // Success!
    }
    // CAS failed (another thread modified head), retry
}

// Now we have successfully reserved the slot
// head contains the OLD value (before increment)
```

CAS Explanation:

```
bool atomic_compare_exchange_weak(atomic_uint64_t* obj,
                                  uint64_t* expected,
                                  uint64_t desired,
                                  memory_order success,
                                  memory_order failure) {

    // Atomically:
    if (*obj == *expected) {
        *obj = desired;
        return true; // Success
    } else {
        *expected = *obj; // Update expected with current value
        return false; // Failure
    }
}
```

```

    }
}

```

Key Points: - Use CAS for conditional updates - Check condition before modification - Loop on CAS failure (another thread modified) - Use **weak** for loops, **strong** for single attempts - Success ordering usually `acq_rel`, failure usually `acquire`

Rule CS-3: Apply Proper Memory Ordering

Principle: Use `acquire`/release semantics for synchronization.

Memory Ordering Overview:

Ordering	Usage	Guarantees
<code>relaxed</code>	Counters, statistics	No synchronization, just atomicity
<code>acquire</code>	Load operations	Reads after cannot move before
<code>release</code>	Store operations	Writes before cannot move after
<code>acq_rel</code>	RMW operations	Both <code>acquire</code> and <code>release</code>
<code>seq_cst</code>	When in doubt	Strongest ordering (slower)

WRONG:

```

// Publish data without proper ordering
data->field = value;
atomic_store(&data->ready, true, memory_order_relaxed); // Can reorder!

// Consumer:
if (atomic_load(&data->ready, memory_order_relaxed)) { //
    process(data->field); // May see uninitialized data!
}

```

CORRECT:

```

// Producer: Use release to publish
data->field = value;
atomic_store(&data->ready, true, memory_order_release); //

// Consumer: Use acquire to synchronize
if (atomic_load(&data->ready, memory_order_acquire)) { //
    process(data->field); // Guaranteed to see initialized data
}

```

Common Patterns:

Producer-Consumer:

```

// Producer:
write_data();
atomic_store(&ready_flag, true, memory_order_release);

// Consumer:
while (!atomic_load(&ready_flag, memory_order_acquire)) {
    // wait
}
read_data();

```

Mutex Lock:

```
// Lock acquisition:
while (atomic_exchange(&lock, 1, memory_order_acquire) == 1) {
    // spin or yield
}

// Lock release:
atomic_store(&lock, 0, memory_order_release);
```

Reference Counting:

```
// Increment: relaxed (no synchronization needed)
atomic_fetch_add(&refcount, 1, memory_order_relaxed);

// Decrement: release (if last reference, must sync with other decrements)
if (atomic_fetch_sub(&refcount, 1, memory_order_release) == 1) {
    atomic_thread_fence(memory_order_acquire); // Sync before destruction
    destroy_object();
}
```

Key Points: - Use acquire for loads, release for stores - Use acq_rel for RMW operations - Use relaxed for pure counters/statistics - Use seq_cst when ordering unclear (safest, slowest) - Document memory ordering choices

Rule CS-4: Write Data Before Publishing Pointers

Principle: Ensure data is fully initialized before making it visible.

WRONG:

```
// Publish head before data written - race!
uint64_t head = atomic_fetch_add(&ring->head, 1, memory_order_relaxed);
uint64_t index = head & MASK;

// Head now visible to consumers!
// But message not yet initialized!

ring->messages[index] = msg; // Too late!
ring->ready[index] = true;
```

CORRECT:

```
// Reserve slot
uint64_t head = atomic_fetch_add(&ring->head, 1, memory_order_acquire);
uint64_t index = head & MASK;

// Write message BEFORE publishing
ring->messages[index] = msg;

// Publish with release semantics
atomic_store(&ring->ready[index], true, memory_order_release); //

// Consumer:
if (atomic_load(&ring->ready[index], memory_order_acquire)) {
    msg = ring->messages[index]; // Guaranteed to see initialized message
}
```

Key Points: - Write data first - Publish pointer/flag last with release - Consumer loads flag with acquire - Ensures consumer sees fully initialized data - Test with ThreadSanitizer

Rule CS-5: Provide Scheduler Reference to Fibers/Threads

Principle: Cooperative schedulers need explicit yield points.

WRONG:

```
// Fiber completes but can't yield back
static void fiber_entry_wrapper(void) {
    fiber_t* fiber = g_current_fiber;
    fiber->entry(fiber->arg);
    fiber->state = FIBER_STATE_DEAD;

    // No way to yield back to scheduler!
    // Scheduler never regains control!
    while (1) {
        __asm__ __volatile__("pause");
    }
}
```

CORRECT:

```
// Store scheduler reference in thread-local storage
static __thread void* g_current_scheduler = NULL;

void fiber_set_scheduler(void* scheduler) {
    g_current_scheduler = scheduler;
}

static void fiber_entry_wrapper(void) {
    fiber_t* fiber = g_current_fiber;
    fiber->entry(fiber->arg);
    fiber->state = FIBER_STATE_DEAD;

    // Yield back to scheduler
    if (g_current_scheduler) {
        fiber_scheduler_yield(g_current_scheduler);
    }

    // Fallback (should never reach)
    while (1) {
        __asm__ __volatile__("pause");
    }
}

// In scheduler:
fiber_set_scheduler(scheduler);
fiber_switch_to(fiber);
```

Key Points: - Provide scheduler reference to execution contexts - Use thread-local storage for per-thread schedulers - Always yield back after completion - Handle completion path carefully - Test fiber lifecycle thoroughly

Rule CS-6: Test Concurrency Thoroughly

Principle: Race conditions are non-deterministic; stress testing is essential.

Testing Strategy:

```
void test_concurrent_queue() {
    Queue* q = queue_create();

    // Multiple producers
    #define NUM_PRODUCERS 4
    #define ITEMS_PER_PRODUCER 10000

    pthread_t producers[NUM_PRODUCERS];
    for (int i = 0; i < NUM_PRODUCERS; i++) {
        pthread_create(&producers[i], NULL, producer_thread, q);
    }

    // Multiple consumers
    #define NUM_CONSUMERS 4
    pthread_t consumers[NUM_CONSUMERS];
    for (int i = 0; i < NUM_CONSUMERS; i++) {
        pthread_create(&consumers[i], NULL, consumer_thread, q);
    }

    // Wait for completion
    for (int i = 0; i < NUM_PRODUCERS; i++) {
        pthread_join(producers[i], NULL);
    }
    for (int i = 0; i < NUM_CONSUMERS; i++) {
        pthread_join(consumers[i], NULL);
    }

    // Verify correctness
    assert(items_produced == items_consumed);
    assert(no_duplicates);
    assert(no_lost_items);
}
```

Testing Checklist: - [] Multiple producers and consumers - [] High contention scenarios - [] Random delays to vary interleavings - [] Verify no duplicates - [] Verify no lost items - [] Run with ThreadSanitizer - [] Stress test for extended periods (hours) - [] Test on different architectures

Tools: - **ThreadSanitizer (TSan):** Detects data races - **Helgrind:** Valgrind's thread checker - **Stress test harness:** Custom concurrent stress tests

Error Handling Best Practices

Rule EH-1: Always Check Return Values

Principle: Never ignore return values from functions that can fail.

WRONG:

```
void task_unblock(struct task *task) {
    task_set_state(task, TASK_BLOCKED, TASK_READY);
}
```

```

    struct cpu_runqueue *rq = get_current_runqueue();
    runqueue_enqueue(rq, task); // Ignores return value!

    // Always increments counter, even if enqueue failed!
    atomic_fetch_add(&num_running_tasks, 1); //
}

```

CORRECT:

```

void task_unblock(struct task *task) {
    if (!task_set_state(task, TASK_BLOCKED, TASK_READY)) {
        return; // State transition failed
    }

    struct cpu_runqueue *rq = get_current_runqueue();
    int result = runqueue_enqueue(rq, task);

    if (result == 0) { // Check success
        atomic_fetch_add(&num_running_tasks, 1);
    } else {
        // Revert state on failure
        task_set_state(task, TASK_READY, TASK_BLOCKED);
        log_error("Failed to enqueue task: queue full");
    }
}

```

Key Points: - Check all return values - Handle errors appropriately - Revert changes on failure - Log errors for debugging - Don't ignore return values even in "can't fail" cases

Rule EH-2: Maintain State Consistency on Failure

Principle: Revert partial changes when operations fail.

WRONG:

```

int complex_operation() {
    state->phase = 1;
    state->counter++;

    if (sub_operation_1() < 0) {
        return -1; // State left inconsistent!
    }

    state->phase = 2;
    if (sub_operation_2() < 0) {
        return -1; // State inconsistent!
    }

    return 0;
}

```

CORRECT:

```

int complex_operation() {
    // Save original state

```

```

int original_phase = state->phase;
int original_counter = state->counter;

state->phase = 1;
state->counter++;

if (sub_operation_1() < 0) {
    // Revert changes
    state->phase = original_phase;
    state->counter = original_counter;
    return -1; // State consistent
}

state->phase = 2;
if (sub_operation_2() < 0) {
    // Revert all changes
    state->phase = original_phase;
    state->counter = original_counter;
    // May need to undo sub_operation_1 effects
    undo_sub_operation_1();
    return -1; // State consistent
}

return 0;
}

```

Alternative: Use Cleanup Handlers

```

int complex_operation() {
    bool phase1_done = false;
    bool phase2_done = false;
    int result = -1;

    if (sub_operation_1() < 0) {
        goto cleanup;
    }
    phase1_done = true;

    if (sub_operation_2() < 0) {
        goto cleanup;
    }
    phase2_done = true;

    result = 0; // Success

cleanup:
    if (!phase2_done && phase1_done) {
        undo_sub_operation_1();
    }
    return result;
}

```

Key Points: - Save original state before modifications - Revert changes on failure - Use goto cleanup pattern for complex cleanup - Ensure cleanup handlers are safe to call multiple times - Test error paths thoroughly

Rule EH-3: Use Error Codes Consistently

Principle: Establish and follow consistent error code conventions.

Error Code Convention:

```
// Return 0 on success, negative error codes on failure
#define SUCCESS 0
#define -EINVAL    -1    // Invalid argument
#define -ENOMEM    -2    // Out of memory
#define -EAGAIN    -3    // Try again
#define -EBUSY     -4    // Resource busy
#define -ENOENT    -5    // No such entry
#define -EEXIST    -6    // Already exists
// ... etc
```

WRONG:

```
// Inconsistent error returns
int func1() {
    if (error) return -1; // Generic error
    return 0;
}

int func2() {
    if (error) return 1; // Non-zero for error? Confusing!
    return 0;
}

void* func3() {
    if (error) return NULL; // Can't distinguish error types
    return ptr;
}
```

CORRECT:

```
// Consistent error codes
int func1() {
    if (invalid_arg) return -EINVAL;
    if (out_of_memory) return -ENOMEM;
    return SUCCESS; // 0
}

int func2() {
    if (invalid_arg) return -EINVAL;
    return SUCCESS;
}

// For pointer-returning functions, use out parameter for error
void* func3(int* error_out) {
    if (invalid_arg) {
        if (error_out) *error_out = -EINVAL;
        return NULL;
    }
    if (out_of_memory) {
        if (error_out) *error_out = -ENOMEM;
        return NULL;
    }
}
```

```

    }
    if (error_out) *error_out = SUCCESS;
    return ptr;
}

```

Key Points: - 0 for success, negative for errors - Use descriptive error codes (not just -1) - Document error codes in function comments - For pointers, use out parameter for error code - Be consistent across the codebase

Rule EH-4: Add Assertions for Invariants

Principle: Use assertions to catch violations of assumptions.

```
#include <assert.h>
```

```

void process_request(Request* req, int size) {
    // Document and assert preconditions
    assert(req != NULL);           // Never NULL
    assert(size > 0);              // Positive size
    assert(size <= MAX_SIZE);      // Within bounds
    assert(req->state == READY);   // Correct state

    // ... implementation ...

    // Assert postconditions
    assert(req->state == PROCESSED);
}

```

```

// In debug builds: assertions active
// In release builds: assertions compiled out

```

When to Use Assertions: - Preconditions (function inputs) - Postconditions (function outputs) - Invariants (data structure consistency) - “Can’t happen” conditions - Internal logic checks

When NOT to Use Assertions: - Validating external input (use runtime checks) - Checking resource availability (use error codes) - Conditions that can legitimately occur - Side effects (assertions may be compiled out!)

WRONG:

```

// Never put side effects in assertions!
assert(increment_counter() == SUCCESS); // Won't run in release build!

```

CORRECT:

```

int result = increment_counter();
assert(result == SUCCESS); // Safe

```

I/O and File System Best Practices

Rule IO-1: Honor Intra-Sector Offsets

Principle: Always account for offsets within sectors/pages.

WRONG:

```

// Calculate sector but ignore offset within sector
uint64_t sector = offset / sector_size;
uint32_t sector_offset = offset % sector_size; // Calculated but not used!

// Read directly into output buffer
read_sector(device, sector, buffer, sectors); // Wrong data if offset != 0

```

CORRECT:

```

uint64_t sector = offset / sector_size;
uint32_t sector_offset = offset % sector_size;

if (sector_offset != 0 || size < sector_size) {
    // Unaligned read - use staging buffer
    uint32_t sectors_needed = (sector_offset + size + sector_size - 1) / sector_size;
    uint8_t* temp_buf = alloca(sectors_needed * sector_size);

    read_sector(device, sector, temp_buf, sectors_needed);

    // Copy from correct offset
    memcpy(buffer, temp_buf + sector_offset, size); //
} else {
    // Aligned read - direct
    read_sector(device, sector, buffer, size / sector_size);
}

```

Key Points: - Calculate AND use intra-sector/page offsets - Use staging buffer for unaligned I/O - Optimize aligned I/O separately - Test with various offsets (0, middle, near end)

Rule IO-2: Query Device Properties

Principle: Don't assume device characteristics; query them.

WRONG:

```

#define SECTOR_SIZE 512 // Assumes 512-byte sectors

// Read operation
int read_blocks(uint64_t block, void* buf, size_t count) {
    uint64_t offset = block * SECTOR_SIZE;
    return device_read(offset, buf, count * SECTOR_SIZE);
}

```

CORRECT:

```

struct block_device {
    uint32_t sector_size; // Query from device
    uint32_t block_size; // May differ from sector size
    uint64_t total_sectors;
    // ...
};

int block_device_init(struct block_device* dev) {
    // Query device properties
    dev->sector_size = query_sector_size(dev);
    dev->block_size = query_block_size(dev);
}

```

```

    dev->total_sectors = query_capacity(dev) / dev->sector_size;

    return 0;
}

int read_blocks(struct block_device* dev, uint64_t block,
               void* buf, size_t count) {
    uint64_t offset = block * dev->block_size;
    return device_read(dev, offset, buf, count * dev->block_size);
}

```

Device Properties to Query: - Sector size (512, 4096, etc.) - Block size (filesystem block size) - Alignment requirements - Capacity - Read/write granularity - DMA capabilities

Rule IO-3: Validate Buffer Sizes

Principle: Ensure buffers are large enough for operations.

WRONG:

```

// Fixed-size buffer - may be too small
uint8_t buf[512];
read_file(file, buf, user_requested_size); // Buffer overflow if size > 512!

```

CORRECT:

```

size_t read_file(File* file, void* buffer, size_t size, size_t buffer_capacity) {
    // Validate buffer size
    if (size > buffer_capacity) {
        log_error("Buffer too small: need %zu, have %zu", size, buffer_capacity);
        return -EINVAL;
    }

    // Clip to file size
    if (file->offset + size > file->size) {
        size = file->size - file->offset;
    }

    return do_read(file, buffer, size);
}

// Caller:
uint8_t buf[1024];
size_t bytes_read = read_file(file, buf, requested_size, sizeof(buf));

```

Key Points: - Pass buffer capacity as parameter - Validate size capacity - Clip to file/device size - Return actual bytes read - Document buffer requirements

Algorithm Design Best Practices

Rule AD-1: Avoid Circular Dependencies

Principle: Design call graphs without cycles.

WRONG:

```

// Circular dependency leads to infinite recursion
int mbh_add(Number* a, Number* b) {
    // Convert to same base if needed
    if (a->base != b->base) {
        convert_base(a, b->base);
    }
    return mbh_add_fast(a, b); // Call fast path
}

int mbh_add_fast(Number* a, Number* b) {
    if (a->sign != b->sign) {
        return mbh_add(a, b); // Infinite recursion!
    }
    // ... fast path ...
}

```

CORRECT:

```

// Complete implementation without mutual recursion
int mbh_add_fast(Number* a, Number* b) {
    if (a->sign == b->sign) {
        // Add magnitudes
        return add_magnitudes(a, b);
    } else {
        // Complete implementation for different signs
        int cmp = compare_magnitudes(a, b);
        if (cmp == 0) {
            return zero();
        } else if (cmp > 0) {
            Number* result = subtract_magnitudes(a, b);
            result->sign = a->sign;
            return result;
        } else {
            Number* result = subtract_magnitudes(b, a);
            result->sign = b->sign;
            return result;
        }
    }
}

int mbh_add(Number* a, Number* b) {
    // Convert if needed, then call complete implementation
    if (a->base != b->base) {
        convert_base(a, b->base);
    }
    return mbh_add_fast(a, b); // No recursion back
}

```

Prevention Strategies: - Draw call graphs before implementing - Use tools to detect cycles (static analysis)
 - Implement complete algorithms in leaf functions - Factor out common operations - Test with all input combinations

Rule AD-2: Handle All Edge Cases

Principle: Explicitly handle boundary conditions and special cases.

```
int divide(int a, int b, int* result) {
    // Edge case: division by zero
    if (b == 0) {
        log_error("Division by zero");
        return -EINVAL;
    }

    // Edge case: INT_MIN / -1 overflows
    if (a == INT_MIN && b == -1) {
        log_error("Division overflow");
        return -EOVERFLOW;
    }

    // Normal case
    *result = a / b;
    return SUCCESS;
}
```

Common Edge Cases: - Empty data structures - Single-element data structures - Maximum capacity - Null pointers - Zero values - Negative values - Boundary values (MIN, MAX) - Overflow/underflow - Special floating-point values (NaN, Inf)

Testing Edge Cases:

```
// Test empty
test_queue_empty();

// Test single element
test_queue_single_element();

// Test full capacity
test_queue_full();

// Test boundary values
test_operation(INT_MIN);
test_operation(INT_MAX);
test_operation(0);
test_operation(-1);

// Test null pointers
test_operation(NULL);
```

Rule AD-3: Design for Testability

Principle: Write code that is easy to test.

WRONG:

```
// Hard to test: tightly coupled, hidden dependencies
void process_data() {
    Data* data = read_from_global_database(); // Hidden dependency
    transform_data(data);
}
```

```
    write_to_global_database(data); // Can't test without database
}
```

CORRECT:

```
// Easy to test: explicit dependencies, injectable
void process_data(Database* db, Transformer* transform) {
    Data* data = db->read(db);
    transform->transform(transform, data);
    db->write(db, data);
}

// Test with mock database
void test_process_data() {
    MockDatabase mock_db = create_mock_database();
    MockTransformer mock_transform = create_mock_transformer();

    // Set up expectations
    mock_db_expect_read(&mock_db, test_data);
    mock_transform_expect_call(&mock_transform);
    mock_db_expect_write(&mock_db, expected_result);

    // Execute
    process_data(&mock_db.base, &mock_transform.base);

    // Verify
    assert(mock_db_verify(&mock_db));
    assert(mock_transform_verify(&mock_transform));
}
```

Testability Guidelines: - Inject dependencies (don't hardcode globals) - Return values instead of side effects when possible - Separate I/O from logic - Keep functions small and focused - Avoid hidden state - Make internal functions testable

Code Quality and Maintainability

Rule CQ-1: Write Self-Documenting Code

Principle: Code should be readable and understandable without excessive comments.

WRONG:

```
// Obscure code requiring comments
int f(int x, int y) { // Cryptic names
    int z = (x + y - 1) / y * y; // Magic formula
    return z;
}
```

CORRECT:

```
// Self-documenting code
int round_up_to_multiple(int value, int multiple) {
    // Use descriptive names that explain intent
    int rounded = ((value + multiple - 1) / multiple) * multiple;
    return rounded;
}
```

```
// Alternative with even clearer implementation
int round_up_to_multiple(int value, int multiple) {
    int remainder = value % multiple;
    if (remainder == 0) {
        return value;
    }
    return value + (multiple - remainder);
}
```

Self-Documentation Checklist: - [] Descriptive variable names - [] Descriptive function names - [] Clear control flow - [] Appropriate abstraction level - [] Consistent naming conventions - [] Type clarity (use typedef for complex types)

Rule CQ-2: Document Complex Logic

Principle: Add comments for non-obvious implementations.

```
/**
 * @brief Atomically dequeue task from run queue using CAS loop
 *
 * This function uses compare-and-swap to avoid pre-incrementing the head
 * pointer before verifying the queue is non-empty. Pre-incrementing would
 * cause concurrent producers to think the queue is full (head > tail).
 *
 * @param rq Run queue to dequeue from
 * @return Task pointer on success, NULL if queue empty
 */
struct task* runqueue_dequeue(struct cpu_runqueue* rq) {
    while (1) {
        uint64_t head = atomic_load(&rq->head, memory_order_acquire);
        uint64_t tail = atomic_load(&rq->tail, memory_order_acquire);

        // Check if queue is empty BEFORE incrementing
        if (head >= tail) {
            return NULL;
        }

        // Try to atomically increment head - only if still equal to expected value
        // If another thread modified head, CAS fails and we retry
        if (atomic_compare_exchange_weak(&rq->head, &head, head + 1,
                                         memory_order_acq_rel,
                                         memory_order_acquire)) {
            break; // Success - we reserved this slot
        }
        // CAS failed, retry with updated head value
    }

    // At this point, we successfully reserved the slot at index 'head'
    uint64_t index = head & RUNQUEUE_MASK;
    struct task* task = rq->tasks[index];
    rq->tasks[index] = NULL;

    return task;
}
```

```
}
```

What to Document: - Why (rationale for design decisions) - Concurrency considerations - Memory ordering choices - Non-obvious algorithms - Assumptions and invariants - Limitations and edge cases

What NOT to Document: - What (code should be self-explanatory) - Obvious operations - Redundant information

Rule CQ-3: Use Consistent Naming Conventions

BDI Project Naming Conventions:

```
// Constants: UPPER_SNAKE_CASE
#define MAX_TASKS 1024
#define CACHE_LINE_SIZE 64

// Types: snake_case_t
typedef struct task task_t;
typedef struct cpu_runqueue cpu_runqueue_t;

// Functions: module_operation (snake_case)
void task_create();
void runqueue_enqueue();
int pmm_alloc_page();

// Global variables: g_prefix
cpu_runqueue_t g_runqueue;
scheduler_t g_scheduler;

// Static (file-local): s_prefix or no prefix
static lock_t s_global_lock;

// Local variables: snake_case
int task_count;
void* page_ptr;

// Structure members: snake_case
struct task {
    int task_id;
    state_t task_state;
    void* stack_pointer;
};

// Enumeration values: PREFIX_VALUE
typedef enum {
    TASK_READY,
    TASK_RUNNING,
    TASK_BLOCKED,
    TASK_DEAD
} task_state_t;
```

Rule CQ-4: Keep Functions Small and Focused

Principle: Each function should do one thing well.

Guidelines: - < 50 lines preferred - < 100 lines maximum (with exceptions) - One level of abstraction per function - Single responsibility principle - Extract helper functions

WRONG:

```
// 200-line monster function doing everything
int process_request(Request* req) {
    // Parse request (50 lines)
    // Validate request (50 lines)
    // Execute operation (50 lines)
    // Format response (50 lines)
    // Too long, does too much
}
```

CORRECT:

```
int process_request(Request* req) {
    ParsedRequest parsed;
    if (parse_request(req, &parsed) < 0) {
        return -EINVAL;
    }

    if (validate_request(&parsed) < 0) {
        return -EINVAL;
    }

    Result result;
    if (execute_operation(&parsed, &result) < 0) {
        return -EIO;
    }

    return format_response(&result, req->response_buffer);
}

// Each helper function is small and focused
static int parse_request(Request* req, ParsedRequest* out) { /* ... */ }
static int validate_request(ParsedRequest* req) { /* ... */ }
static int execute_operation(ParsedRequest* req, Result* out) { /* ... */ }
static int format_response(Result* result, char* buffer) { /* ... */ }
```

Testing and Validation Strategies

Rule TV-1: Write Tests First (TDD)

Test-Driven Development Process:

1. Write failing test

```
void test_queue_enqueue_dequeue() {
    Queue* q = queue_create(10);

    // Test doesn't compile yet - queue_enqueue doesn't exist
    assert(queue_enqueue(q, (void*)1) == 0);
}
```

```

    assert(queue_enqueue(q, (void*)2) == 0);

    void* item;
    assert(queue_dequeue(q, &item) == 0);
    assert(item == (void*)1);
    assert(queue_dequeue(q, &item) == 0);
    assert(item == (void*)2);

    queue_destroy(q);
}

```

2. Implement minimal code to make it compile

```

int queue_enqueue(Queue* q, void* item) {
    return -1; // Stub - returns failure
}

int queue_dequeue(Queue* q, void** item) {
    return -1; // Stub - returns failure
}

```

3. Run test - should fail

4. Implement functionality

```

int queue_enqueue(Queue* q, void* item) {
    if (queue_full(q)) return -EAGAIN;
    q->items[q->tail] = item;
    q->tail = (q->tail + 1) % q->capacity;
    q->count++;
    return 0;
}

int queue_dequeue(Queue* q, void** item) {
    if (queue_empty(q)) return -EAGAIN;
    *item = q->items[q->head];
    q->head = (q->head + 1) % q->capacity;
    q->count--;
    return 0;
}

```

5. Run test - should pass

6. Refactor if needed

Benefits: - Forces thinking about API before implementation - Ensures testability - Provides immediate feedback - Creates regression test suite - Documents expected behavior

Rule TV-2: Test Error Paths

Principle: Test failure scenarios, not just success paths.

```

void test_queue_operations() {
    Queue* q = queue_create(2); // Small capacity

    // Success cases
    assert(queue_enqueue(q, (void*)1) == 0);
}

```

```

assert(queue_enqueue(q, (void*)2) == 0);

// Error case: full queue
assert(queue_enqueue(q, (void*)3) == -EAGAIN); // Test error path
assert(queue_count(q) == 2); // Verify state unchanged

void* item;
assert(queue_dequeue(q, &item) == 0);
assert(queue_dequeue(q, &item) == 0);

// Error case: empty queue
assert(queue_dequeue(q, &item) == -EAGAIN); // Test error path
assert(queue_count(q) == 0); // Verify state unchanged

queue_destroy(q);
}

```

Error Paths to Test: - Null pointer arguments - Out of bounds indices - Full/empty data structures - Out of memory conditions - Invalid state transitions - Concurrent access conflicts

Rule TV-3: Use Sanitizers

AddressSanitizer (ASan):

```

# Compile with AddressSanitizer
gcc -fsanitize=address -g -O1 code.c -o program

# Run
./program

# ASan detects:
# - Buffer overruns
# - Use-after-free
# - Double-free
# - Memory leaks
# - Use of uninitialized memory

```

ThreadSanitizer (TSan):

```

# Compile with ThreadSanitizer
gcc -fsanitize=thread -g -O1 code.c -o program

# TSan detects:
# - Data races
# - Deadlocks
# - Thread leaks
# - Improper atomic usage

```

UndefinedBehaviorSanitizer (UBSan):

```

# Compile with UBSan
gcc -fsanitize=undefined -g -O1 code.c -o program

# UBSan detects:
# - Integer overflow
# - Null pointer dereference

```

```
# - Misaligned access
# - Division by zero
```

MemorySanitizer (MSan):

```
# Compile with MSan
clang -fsanitize=memory -g -O1 code.c -o program
```

```
# MSan detects:
# - Use of uninitialized memory
```

All Sanitizers:

```
# Combine multiple (not all can be combined)
gcc -fsanitize=address -fsanitize=undefined -g -O1 code.c -o program
```

Rule TV-4: Achieve High Test Coverage

Coverage Goals: - **Minimum:** 70% line coverage - **Target:** 80% line coverage - **Critical code:** 100% coverage (memory management, concurrency, security)

Measuring Coverage:

```
# GCC with gcov
gcc -fprofile-arcs -ftest-coverage code.c -o program
./program
gcov code.c
lcov --capture --directory . --output-file coverage.info
genhtml coverage.info --output-directory coverage_html

# Clang with llvm-cov
clang -fprofile-instr-generate -fcoverage-mapping code.c -o program
LLVM_PROFILE_FILE="code.profraw" ./program
llvm-profdata merge -sparse code.profraw -o code.profdata
llvm-cov show program -instr-profile=code.profdata
```

Coverage Types: - **Line coverage:** Percentage of lines executed - **Branch coverage:** Percentage of branches taken - **Function coverage:** Percentage of functions called - **Path coverage:** Percentage of execution paths taken

Rule TV-5: Fuzz Test Parsers and Inputs

Fuzzing Strategy:

```
// Fuzz test function
void fuzz_parser(uint8_t* data, size_t size) {
    Parser* parser = parser_create();

    // Feed random data
    Result result = parser_parse(parser, data, size);

    // Should not crash, regardless of input
    // May return error, but must not have undefined behavior

    parser_destroy(parser);
}
```

```
// Run with AFL (American Fuzzy Lop)
// Or libFuzzer
```

Using libFuzzer:

```
#include <stdint.h>
#include <stddef.h>

// Fuzzing entry point
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    fuzz_parser(data, size);
    return 0;
}

# Compile with libFuzzer
clang -fsanitize=fuzzer,address -g code.c -o fuzzer

# Run
./fuzzer

# libFuzzer will:
# - Generate random inputs
# - Mutate inputs to explore code paths
# - Detect crashes and hangs
# - Save crashing inputs to disk
```

What to Fuzz: - Parsers (bytecode, config files, protocols) - File format readers - Network input handlers
- Compression/decompression - Cryptographic functions

Security Best Practices

Rule SEC-1: Validate All External Input

Principle: Never trust external input; always validate.

```
int handle_network_packet(uint8_t* data, size_t size) {
    // Validate minimum size
    if (size < sizeof(PacketHeader)) {
        log_error("Packet too small: %zu bytes", size);
        return -EINVAL;
    }

    PacketHeader* header = (PacketHeader*)data;

    // Validate header fields
    if (header->version != PROTOCOL_VERSION) {
        log_error("Invalid protocol version: %u", header->version);
        return -EINVAL;
    }

    if (header->type >= PACKET_TYPE_MAX) {
        log_error("Invalid packet type: %u", header->type);
        return -EINVAL;
    }
}
```

```

if (header->length > MAX_PACKET_LENGTH) {
    log_error("Packet too large: %u bytes", header->length);
    return -EINVAL;
}

if (size < sizeof(PacketHeader) + header->length) {
    log_error("Packet truncated: have %zu, need %zu",
        size, sizeof(PacketHeader) + header->length);
    return -EINVAL;
}

// Now safe to process
return process_packet(header, data + sizeof(PacketHeader), header->length);
}

```

Validation Checklist: - [] Minimum/maximum size - [] Value ranges - [] Enum values in range - [] Pointer validity (not NULL) - [] String null termination - [] Array indices in bounds - [] Integer overflow in calculations

Rule SEC-2: Prevent Integer Overflow

Principle: Check for overflow before performing arithmetic.

WRONG:

```

void* allocate_array(size_t count, size_t element_size) {
    size_t total_size = count * element_size; // May overflow!
    return malloc(total_size);
}

```

CORRECT:

```

void* allocate_array(size_t count, size_t element_size) {
    // Check for overflow before multiplication
    if (count > 0 && element_size > SIZE_MAX / count) {
        log_error("Allocation would overflow: %zu * %zu", count, element_size);
        return NULL;
    }

    size_t total_size = count * element_size;
    return malloc(total_size);
}

// Alternative: use compiler builtin
void* allocate_array_builtin(size_t count, size_t element_size) {
    size_t total_size;
    if (__builtin_mul_overflow(count, element_size, &total_size)) {
        log_error("Allocation would overflow");
        return NULL;
    }

    return malloc(total_size);
}

```

Overflow Detection:

```

// Addition overflow check
if (a > UINT64_MAX - b) {
    // Would overflow
}

// Multiplication overflow check
if (a > 0 && b > UINT64_MAX / a) {
    // Would overflow
}

// Use compiler builtins (GCC/Clang)
if (__builtin_add_overflow(a, b, &result)) {
    // Overflowed
}
if (__builtin_mul_overflow(a, b, &result)) {
    // Overflowed
}

```

Rule SEC-3: Use Safe String Functions

Principle: Prefer bounds-checked string functions.

WRONG:

```

char buf[64];
strcpy(buf, user_input); // Buffer overflow if input > 63 chars!
strcat(buf, suffix);     // May overflow
sprintf(buf, "%s%s", prefix, suffix); // May overflow

```

CORRECT:

```

char buf[64];

// Use bounds-checked versions
strncpy(buf, user_input, sizeof(buf) - 1);
buf[sizeof(buf) - 1] = '\0'; // Ensure null termination

// Or better: use strncpy (if available)
strncpy(buf, user_input, sizeof(buf));

// For concatenation
strlcat(buf, suffix, sizeof(buf));

// For formatting
snprintf(buf, sizeof(buf), "%s%s", prefix, suffix);

```

Safe String Functions: - `strncpy()` → but remember to null-terminate! - `strncpy()` → safer, always null-terminates - `strlcat()` → safer concatenation - `snprintf()` → bounds-checked formatting - Avoid: `strcpy()`, `strcat()`, `sprintf()`, `gets()`

Rule SEC-4: Clear Sensitive Data

Principle: Overwrite sensitive data after use.

```

void secure_operation(const char* password) {
    char password_copy[256];

    // Copy password for processing
    strcpy(password_copy, password, sizeof(password_copy));

    // Use password
    int result = authenticate(password_copy);

    // Clear sensitive data
    explicit_bzero(password_copy, sizeof(password_copy));
    // OR:
    // memset_s(password_copy, sizeof(password_copy), 0, sizeof(password_copy));
    // OR:
    // volatile char* p = password_copy;
    // for (size_t i = 0; i < sizeof(password_copy); i++) {
    //     p[i] = 0;
    // }

    return result;
}

```

Why Not Regular memset?

```
memset(password_copy, 0, sizeof(password_copy)); // May be optimized away!
```

```

// Compiler may remove this if password_copy is not used after
// Use explicit_bzero() or memset_s() or volatile pointer

```

Performance Optimization Guidelines

Rule PERF-1: Measure Before Optimizing

Principle: Profile to find bottlenecks; don't guess.

Use perf on Linux

```
perf record -g ./program
perf report
```

Use gprof

```
gcc -pg code.c -o program
./program
gprof program gmon.out > analysis.txt
```

Use Valgrind callgrind

```
valgrind --tool=callgrind ./program
kcachegrind callgrind.out.*
```

What to Profile: - CPU hotspots (functions taking most time) - Cache misses - Branch mispredictions - Memory allocations - I/O operations

Rule PERF-2: Optimize Cache Usage

Principle: Improve spatial and temporal locality.

Cache-Friendly Patterns:

```
// Poor cache usage (column-major access)
for (int j = 0; j < N; j++) {
    for (int i = 0; i < M; i++) {
        sum += matrix[i][j]; // Cache miss every access
    }
}

// Good cache usage (row-major access)
for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) {
        sum += matrix[i][j]; // Cache-friendly
    }
}

// Align hot data structures to cache lines
struct __attribute__((aligned(64))) HotData {
    atomic_uint64_t counter;
    char padding[64 - sizeof(atomic_uint64_t)];
};

// Group frequently accessed fields together
struct Task {
    // Hot fields (accessed frequently)
    task_state_t state;
    int priority;
    void* stack_ptr;

    // Cold fields (accessed rarely)
    char name[64];
    timestamp_t creation_time;
};
```

Rule PERF-3: Minimize Allocations

Principle: Reduce allocation frequency in hot paths.

WRONG:

```
void process_items(Item* items, size_t count) {
    for (size_t i = 0; i < count; i++) {
        Buffer* buf = malloc(sizeof(Buffer)); // Allocate every iteration
        process_item(&items[i], buf);
        free(buf); // Free every iteration
    }
}
```

CORRECT:

```
void process_items(Item* items, size_t count) {
    // Allocate once outside loop
    Buffer buf;

    for (size_t i = 0; i < count; i++) {
```

```

        process_item(&items[i], &buf);
        reset_buffer(&buf); // Reuse buffer
    }
}

// OR use memory pool
void process_items_pooled(Item* items, size_t count) {
    MemoryPool* pool = pool_create();

    for (size_t i = 0; i < count; i++) {
        Buffer* buf = pool_alloc(pool); // Fast pool allocation
        process_item(&items[i], buf);
        // Pool is reset all at once after loop
    }

    pool_destroy(pool); // Free all at once
}

```

Code Review Checklist

Pre-Review (Author)

Before Submitting: - ☐ Code compiles without warnings - ☐ All tests pass - ☐ AddressSanitizer clean - ☐ ThreadSanitizer clean (if concurrent) - ☐ Code formatted (clang-format) - ☐ Static analysis clean (clang-tidy, cppcheck) - ☐ Documentation updated - ☐ Commit messages descriptive

During Review (Reviewer)

Correctness: - ☐ Algorithm is correct - ☐ Edge cases handled - ☐ Error paths tested - ☐ No undefined behavior - ☐ No memory leaks

Memory Safety: - ☐ Bounds checking correct - ☐ No use-after-free - ☐ No double-free - ☐ Refcounts correct - ☐ Aligned allocations store original pointer

Concurrency: - ☐ Atomic operations used correctly - ☐ Memory ordering appropriate - ☐ No data races - ☐ No deadlocks possible - ☐ Locks acquired in consistent order

Error Handling: - ☐ Return values checked - ☐ Errors propagated correctly - ☐ State consistent on failure - ☐ Resources cleaned up

Testing: - ☐ Tests added for new functionality - ☐ Tests added for bug fixes - ☐ Error paths tested - ☐ Concurrent tests if applicable

Code Quality: - ☐ Readable and maintainable - ☐ Properly documented - ☐ Follows project conventions - ☐ No code duplication - ☐ Functions appropriately sized

Conclusion

This guide consolidates best practices learned from analyzing and fixing 35+ critical bugs in the BDI project. Following these guidelines will help prevent the most common classes of bugs:

Key Takeaways

1. **Memory Safety First**
 - Validate bounds

- Track resources properly
- Use sanitizers
- 2. **Concurrency Correctness**
 - Use atomic operations correctly
 - Apply proper memory ordering
 - Test thoroughly
- 3. **Robust Error Handling**
 - Check all return values
 - Maintain consistency
 - Test error paths
- 4. **Comprehensive Testing**
 - Write tests first
 - Test edge cases
 - Use sanitizers
 - Fuzz test inputs
- 5. **Code Quality**
 - Write readable code
 - Document complexity
 - Follow conventions
 - Review carefully

Continuous Improvement

- Learn from every bug
- Update guidelines as needed
- Share knowledge with team
- Invest in tooling and testing
- Foster a culture of quality

Resources

Tools: - GCC/Clang compilers - AddressSanitizer, ThreadSanitizer, UBSan - Valgrind - Clang-tidy, cp-pcheck - gdb, lldb - perf, gprof

Books: - “Effective C” by Robert C. Seacord - “The Art of Multiprocessor Programming” - “Computer Systems: A Programmer’s Perspective” - “Secure Coding in C and C++”

Standards: - C17/C23 Standard - MISRA C Guidelines - SEI CERT C Coding Standard - NASA C Coding Standards

Guide Version: 1.0

Last Updated: October 10, 2025

Maintainer: BDI Development Team

This guide is a living document. As we encounter new patterns and bugs, we will update it to reflect our learnings. Contributions and suggestions are welcome.